

JAVA 8 vs JAVA 17 vs JAVA 21

Handwritten-style interview study notes for a Java / Spring Boot backend developer

Golden rule: When an interviewer asks "Java 17 features", clarify whether they mean features introduced exactly in 17 or modern features available when moving from Java 8 to Java 17. Many popular "Java 17" items actually arrived in Java 9-16.

1) LTS timeline - remember this first

Java 8 2014	Java 11 2018	Java 17 2021	Java 21 2023
Functional Java Lambdas + Streams	HTTP Client API cleanup	Modern language baseline	Modern concurrency Virtual Threads

2) One-line memory map

- **Java 8** -> functional programming becomes mainstream Java: lambdas, streams, Optional, java.time, CompletableFuture.
- **Java 17** -> cleaner domain modeling + syntax: records, pattern matching for instanceof, switch expressions, text blocks, sealed classes, var, modules, modern HttpClient.
- **Java 21** -> scalable blocking-style concurrency + richer patterns: virtual threads, record patterns, pattern matching for switch, sequenced collections.

3) Best study order for interviews

- First: Java 8 Streams + Lambda + Functional Interfaces + CompletableFuture.
- Second: Java 8 -> 17 migration differences.
- Third: Java 21 Virtual Threads deeply - use case, limitations, CPU-bound vs I/O-bound.
- Finally: pattern matching + record patterns + sequenced collections.

Senior-level habit: Do not only list features. Explain why the feature exists, where you used/would use it, and one trade-off.

JAVA 8 - Functional Java

Released 2014. Still the baseline for many enterprise Java interview questions.

Core features to explain confidently

- **Lambda Expressions** - pass behavior as data; concise replacement for many anonymous classes.
- **Functional Interfaces** - one abstract method. Common: Predicate, Function, Consumer, Supplier.
- **Method References** - shorthand like `String::trim` or `System.out::println`.
- **Stream API** - declarative collection processing: filter / map / flatMap / reduce / collect.
- **Optional** - models presence/absence; do not use it blindly for every field.
- **java.time** - immutable/thread-safe date-time API: LocalDate, Instant, ZonedDateTime, Duration.
- **Default + static methods in interfaces** - evolve interfaces while preserving compatibility.
- **CompletableFuture** - asynchronous composition.
- **Metaspace** - replaced PermGen for class metadata.

Stream example - Java 8 style

```
List<String> activeNames = users.stream()
    .filter(User::isActive)
    .map(User::getName)
    .filter(Objects::nonNull)
    .collect(Collectors.toList());
```

CompletableFuture example

```
CompletableFuture<User> future =
    CompletableFuture.supplyAsync(() -> userClient.fetch(id));

future.thenApply(User::getName)
    .thenAccept(System.out::println)
    .exceptionally(ex -> { log.error("failed", ex); return null; });
```

Java 8 interview traps

- **Stream != collection**. A stream is a lazy processing pipeline.
- **parallelStream()** typically uses the common ForkJoinPool.
- **map()** transforms; **flatMap()** transforms + flattens nested containers/streams.
- **orElse()** evaluates eagerly; **orElseGet()** evaluates lazily.
- **CompletableFuture** is not the same thing as Java 21 Virtual Threads.

30-sec answer: "Java 8 introduced lambdas, functional interfaces and streams, enabling functional-style collection processing. It also added Optional, java.time, default interface methods and CompletableFuture for asynchronous composition."

JAVA 17 - What changes from Java 8?

Java 17 is LTS. The practical upgrade includes features delivered across Java 9 through 17.

Be precise: Java 17 itself finalized sealed classes. Records and pattern matching for instanceof became final in Java 16; text blocks in 15; switch expressions in 14. They are still key differences you gain when upgrading from Java 8 to Java 17.

Most useful features available by Java 17

- **Records** - concise data carriers; generated accessors, equals, hashCode, toString.
- **Pattern matching for instanceof** - combines type check + cast.
- **Sealed classes/interfaces** - explicitly control permitted subtypes.
- **Switch expressions** - switch can return a value; arrow labels reduce fall-through bugs.
- **Text blocks** - readable multi-line JSON / SQL / HTML.
- **var** - local-variable type inference; still statically typed.
- **List.of / Set.of / Map.of** - convenient unmodifiable collection factories.
- **java.net.http.HttpClient** - standardized in Java 11 with sync/async + HTTP/2.
- **Helpful NullPointerExceptions** - more precise diagnostics.
- **Strong encapsulation** of JDK internals - important during migration.

Before vs after: instanceof

```
// Java 8
if (obj instanceof String) {
    String s = (String) obj;
    System.out.println(s.length());
}

// Java 17
if (obj instanceof String s && !s.isBlank()) {
    System.out.println(s.length());
}
```

Records + sealed hierarchy

```
sealed interface Payment permits CardPayment, UpiPayment {}
record CardPayment(String last4) implements Payment {}
record UpiPayment(String vpa) implements Payment {}
```

Switch expression

```
String result = switch (status) {
    case 200, 201 -> "SUCCESS";
    case 404      -> "NOT_FOUND";
    default       -> "OTHER";
};
```

JAVA 17 - Senior migration notes

What an experienced backend developer should mention beyond syntax.

Java 8 -> 17 migration checks

- Upgrade build plugins, test frameworks, bytecode-manipulation libraries and runtime dependencies.
- Check code using JDK-internal APIs or deep reflection; strong encapsulation can break old assumptions.
- Libraries that used to ship inside the JDK may need explicit dependencies after Java 8.
- Re-run performance/load tests. JVM, JIT and GC behavior changed significantly across releases.
- Do not rewrite everything only because a new syntax exists; migrate incrementally and keep behavior tests.

Spring Boot note framework, not JDK

Spring Boot 3 requires Java 17+ and uses Jakarta EE namespaces (`jakarta.*` instead of many `javax.*` APIs). This is a Spring/Jakarta ecosystem concern, not a Java 17 language feature.

Java 17 interview quick answers

- **Record vs normal class?** Great for transparent data carriers; implicitly final; auto accessors + equals/hashCode/toString.
- **Can record extend another class?** No. It already extends `java.lang.Record`, but can implement interfaces.
- **Why sealed classes?** Model a closed domain and restrict inheritance deliberately.
- **Direct subclass of sealed type?** Must be final, sealed or non-sealed (subject to record/enum rules).
- **Is var dynamic typing?** No. Type is inferred at compile time for local variables.
- **Why switch expression?** Returns a value, avoids accidental fall-through, more concise.

30-sec answer: "Compared with Java 8, Java 17 gives a much more expressive language: records, pattern matching for instanceof, switch expressions, text blocks, sealed types and var; plus modern APIs like HttpClient and collection factories. During migration I also check strong encapsulation and dependency compatibility."

JAVA 21 - Modern concurrency

Java 21 is LTS. For backend developers, the standout feature is Virtual Threads.

Stable headline features in Java 21

- **Virtual Threads (JEP 444)** - lightweight JVM-managed threads for high-throughput blocking I/O.
- **Pattern Matching for switch (JEP 441)** - switch on types/patterns, with better null handling.
- **Record Patterns (JEP 440)** - destructure record components directly.
- **Sequenced Collections (JEP 431)** - uniform first / last / reversed operations.
- **Generational ZGC (JEP 439)** - generational mode for ZGC.

Virtual Threads - example

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    Future<String> a = executor.submit(() -> callServiceA());
    Future<String> b = executor.submit(() -> callServiceB());
    return a.get() + b.get();
}
```

Why virtual threads matter

- Platform threads are relatively expensive; blocking services often need carefully sized pools.
- Virtual threads keep simple thread-per-task blocking code while supporting very high concurrency.
- **Best fit:** REST calls, DB calls, network/file I/O, request handling - workloads that spend time waiting.
- **Not a CPU accelerator:** CPU-bound work remains limited by CPU cores.
- **Java 21 nuance:** long blocking operations while holding certain **synchronized** monitors can pin the carrier thread.

Pattern matching for switch

```
static String describe(Object o) {
    return switch (o) {
        case null        -> "null";
        case Integer i    -> "int=" + i;
        case String s     -> "text=" + s;
        default           -> "other";
    };
}
```

JAVA 21 - More + Quick comparison

Record patterns

```
record Point(int x, int y) {}

if (obj instanceof Point(int x, int y)) {
    System.out.println(x + "," + y);
}
```

Sequenced collections

```
var names = new ArrayList<>(
    List.of("A", "B", "C"));

names.getFirst();
names.getLast();
names.reversed();
```

Preview in Java 21 - say "preview", not stable

- String Templates (preview in Java 21).
- Scoped Values (preview).
- Structured Concurrency (preview).
- Foreign Function & Memory API was still preview in Java 21 - do not claim it was final in 21.

Java 8 vs 17 vs 21 - interview table

Area	Java 8	Java 17	Java 21
Programming style	Lambdas + Streams	Modern concise syntax	Pattern-oriented modern Java
Data carrier	POJO / Lombok	record available	record + record patterns
Type checks	instanceof + cast	pattern instanceof	pattern switch + record patterns
Concurrency	Executor + CompletableFuture	same model + JVM improvements	Virtual Threads
Switch	statement	switch expression available	type pattern matching final
Collections	traditional factories	List.of / Set.of / Map.of	Sequenced Collections
HTTP	URLConnection / 3rd party common	java.net.http.HttpClient	same modern client

Virtual Threads vs CompletableFuture: Virtual threads simplify large numbers of blocking tasks using ordinary synchronous code. CompletableFuture remains useful for explicit asynchronous composition, dependency graphs, callbacks and pipeline-style orchestration.

INTERVIEW REVISION SHEET

Question: "What is the difference between Java 8, Java 17 and Java 21?"

60-sec answer: "Java 8 introduced the functional programming foundation with lambdas, streams, Optional, java.time and CompletableFuture. Moving to Java 17 gives modern language features accumulated through Java 9-17, such as records, pattern matching for instanceof, switch expressions, text blocks, sealed classes, var, modules and HttpClient. Java 21 then adds virtual threads for scalable blocking I/O, record patterns, pattern matching for switch and sequenced collections. For backend systems, the biggest practical changes are cleaner domain modeling in 17 and much simpler high-concurrency I/O code in 21."

Follow-up questions to prepare

- Why choose Java 21 virtual threads instead of increasing a platform-thread pool?
- Virtual thread vs platform thread vs CompletableFuture?
- When should you NOT use parallelStream()?
- Record vs Lombok @Data class?
- Why do sealed class + pattern switch work well together?
- Difference between map() and flatMap() in Stream and Optional?
- Difference between orElse() and orElseGet()?
- What compatibility issues appear in a Java 8 -> Java 17 Spring migration?
- Why is String Template not a stable Java 21 feature?
- Do virtual threads improve CPU-bound processing?

Tiny memory tricks

- **8 = Functional** -> Lambda / Stream / Optional / Time / CompletableFuture.
- **17 = Modeling + readability** -> Record / Sealed / Pattern instanceof / Switch / Text block.
- **21 = Concurrency + patterns** -> Virtual Thread / Pattern switch / Record pattern / Sequenced Collections.

Senior-level example: "Virtual threads fit a service that makes many blocking REST/DB calls. They improve concurrency/throughput for waiting tasks, but they do not make CPU-heavy calculations faster."

ONLINE NOTES + VIDEOS + OFFICIAL DOCS

Existing notes were checked online; technical points in this handout were verified against Oracle / Java documentation.

Existing online study notes / cheat sheets

1. [IT Flashcards - Java 17 \(LTS\) Features Cheat Sheet](#) - compact summary.
2. [IT Flashcards - Java 21 \(LTS\) Features Cheat Sheet](#) - virtual threads, switch/record patterns, sequenced collections.
3. [Reddit - Java 8 to Java 21 Modern Features Cheat Sheet](#) - community overview.
4. [Scribd - Java 8 to Java 21 Feature Comparison Cheat Sheet](#) - broad release-by-release outline (may require sign-in).
5. [TeachToJava - Java 8, 17, 21 features](#) - direct comparison.

YouTube references

Java 8: [Java Brains - Java 8 Lambda Basics 1: Introduction](#)

Java 17: [Official Java - Making Use of Sealed Classes in Java](#)

Patterns: [Official Java - Java 17 to 20 Pattern Matching tutorial \(JEP Cafe #14\)](#)

Java 21 Virtual Threads: [Official Java - Virtual Threads Explained](#)

Java 21 overview: [Official Java - All About Java 21 Launch Event](#) - chaptered long-form session.

Official documentation used to verify this note

[Oracle - What's New in JDK 8](#)

[Oracle - Java 17 Language Changes Summary](#)

[Oracle - Java 21 Language Changes Summary](#)

[Oracle - JDK 21 Release Notes](#)

Accuracy note: Many articles/video titles group features under an LTS version even when the feature became final in an earlier interim release. In interviews, distinguish "introduced in this exact release" from "available when upgrading to this LTS".

Generated: 05 Oct 2026